

Guía del alumno

Módulo 3, Limpieza y análisis exploratorio

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase. Trae el mismo código y las mismas cifras que viste en el laboratorio, con la explicación escrita y la solución de cada ejercicio.

Este módulo se dicta en dos clases. Los bloques 1 a 4 son de la primera y los bloques 5 a 7 de la segunda. Cuando vuelvas a abrir el notebook, ejecútalo desde arriba, porque acá el notebook no solo carga datos, los va modificando, y empezar por la mitad no funciona.

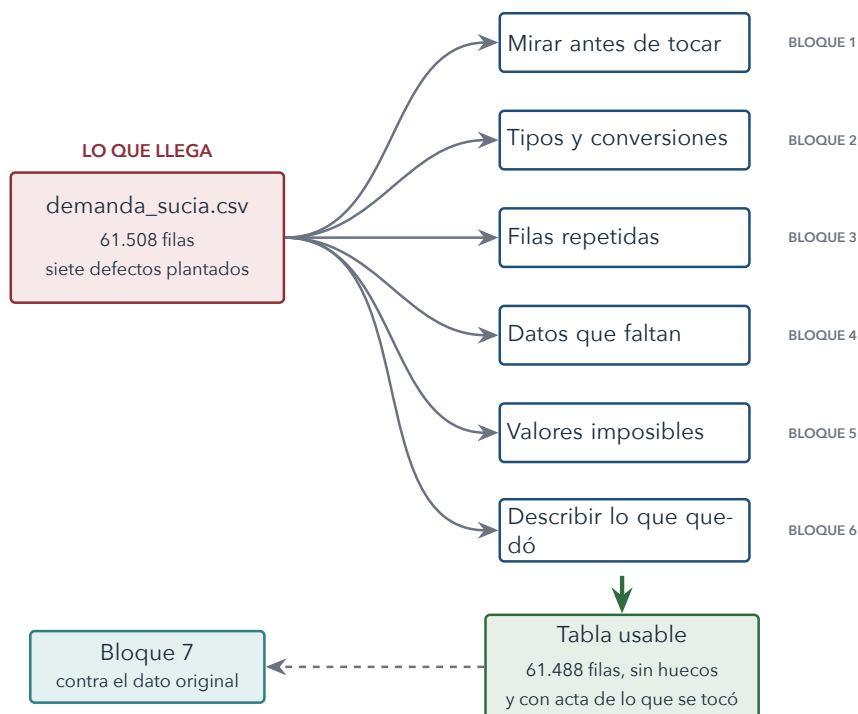
Lo que vas a poder hacer al terminar

- Diagnosticar un archivo nuevo en cinco minutos, antes de calcular nada.
- Arreglar números con coma decimal, fechas en formato chileno y categorías escritas de varias formas.
- Distinguir una fila repetida de una clave repetida, que no se tratan igual.
- Elegir qué hacer con los datos que faltan, sabiendo cuánto cambia el resultado según lo que elijas, y dejar registro de lo que tocaste.
- Separar un valor extremo de uno imposible, y saber por qué el método más sofisticado no siempre es el que encuentra el problema.
- Describir una variable con estadística que se entiende, y auditar tu propia limpieza.

Todas las cifras de esta guía salen de la corrida verificada del notebook con pandas 2.3.3. El archivo sucio se genera dentro del notebook con semilla fija, así que tus cifras son las mismas. Los datos son sintéticos, y donde una cifra del laboratorio se aleja de la realidad del sistema chileno, la guía lo dice.

1. El módulo en una imagen

Llega un archivo con siete defectos plantados. Al final del módulo la tabla queda usable y, más importante, queda escrito qué se tocó y con qué criterio.



La imagen del contenedor dañado

Seguimos en el puerto, y hoy llegó un contenedor dañado.

Algunas cajas llegaron mojadas y no se puede leer lo que traen. Otras vienen con la etiqueta escrita a mano por tres personas distintas. Hay bultos que aparecen dos veces en la lista porque los contaron dos cuadrillas. Faltan cajas que el manifiesto decía que venían. Y hay dos que, según la balanza, pesan diez veces lo que puede pesar una caja de ese tamaño, y una que pesa menos que nada.

El jefe de patio no puede devolver el contenedor. Tiene que dejar la carga utilizable hoy. Entonces hace dos cosas, arregla lo que se puede arreglar y escribe un acta de recepción que dice qué venía dañado y qué se decidió con cada cosa.

Limpiar no es dejar la tabla bonita. Es tomar decisiones sobre datos que no están y dejar por escrito cuáles fueron.

Los siete defectos

una fila del archivo, tal como llega
16/09/2024;12; VALPARAÍSO ;257,45

1. El separador es punto y coma, no coma	todo el archivo
2. La fecha viene como texto y en formato chileno	16/09/2024
3. La región llega con espacios y en mayúsculas	VALPARAÍSO
4. El número usa coma decimal, así que es texto	257,45
5. Hay veinte filas exactamente repetidas	en otras filas
6. Falta el 3 por ciento de los valores, en bloques	celdas vacías
7. Hay picos diez veces más altos y valores negativos	imposibles

Los tres primeros se arreglan en la lectura, con `sep`, `decimal` y el formato de la fecha. Los otros cuatro hay que decidirlos uno por uno, y cada decisión cambia el resultado.

2. Bloque 1, mirar antes de tocar

Qué se vio

La revisión con el camión todavía en el patio. A todo archivo nuevo se le hacen cuatro preguntas antes de calcular nada, y se responden en cuatro líneas.

Cuántas filas tiene y cuántas debería tener. De qué tipo es cada columna. Cuántos valores distintos hay donde debería haber pocos. Y cuántos faltan.

El código, paso a paso

Inicio · El archivo tal como llega

```
# Inicio
import pandas as pd

# A propósito, sin ningún parámetro que arregle nada. Solo el separador,
# porque sin él ni siquiera se ven las columnas.
sucia = pd.read_csv("demanda_sucia.csv", sep=";")

print(sucia.shape)
sucia.head()
```

```
(61508, 4)
   fecha hora  region  mwh
0  01/01/2024  0  Antofagasta  51,95
1  01/01/2024  1  Antofagasta  49,02
2  01/01/2024  2  Antofagasta  48,22
3  01/01/2024  3  Antofagasta  45,57
4  01/01/2024  4  Antofagasta  47,59
```

Se abre sin arreglar nada, solo indicando el separador, porque sin eso ni siquiera se distinguen las columnas. Ya en las primeras filas se ve la fecha en formato chileno y el número con coma decimal.

1.1 Los tipos de cada columna

```
# 1.1
# info muestra el tipo de cada columna y cuántos valores no nulos tiene.
sucia.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61508 entries, 0 to 61507
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   fecha   61508 non-null   object 
 1   hora     61508 non-null   int64  
 2   region   61508 non-null   object 
 3   mwh      59662 non-null   object 
dtypes: int64(1), object(3)
memory usage: 1.9+ MB
```

info responde tres de las cuatro preguntas en una línea. Tres columnas son object, que en pandas quiere decir texto, cuando la fecha debería ser fecha y el megawatt hora debería ser número. Y la cuenta de valores no nulos de la última columna es menor que el total de filas.

1.2 Las filas que deberían ser

```
# 1.2
# 2024 fue bisiesto, así que son 366 días por 24 horas por 7 regiones.
esperadas = 366 * 24 * 7

print("filas esperadas", esperadas)
print("filas del archivo", len(sucia))
print("sobran", len(sucia) - esperadas)
```

```
filas esperadas 61488
filas del archivo 61508
sobran 20
```

366 días por 24 horas por 7 regiones son 61.488 filas. El archivo trae 61.508, o sea veinte de más. Todavía no sabemos cuáles, y ya sabemos que están.

1.3 Las regiones que aparecieron de la nada

```
# 1.3
# nunique cuenta valores distintos. Deberían ser 7 y son bastantes más.
print("regiones distintas", sucia["region"].nunique())
print()
print(sucia["region"].value_counts().head(10))
```

```
regiones distintas 49

region
Valparaíso      8714
Antofagasta     8713
Coquimbo        8707
Metropolitana   8698
Atacama         8693
Los Lagos       8686
Biobío          8683
biobío          26
los lagos       21
BIOBIO          21
Name: count, dtype: int64
```

49 regiones distintas donde deberían ser 7. En el conteo se ve el patrón, las siete reales tienen más de ocho mil filas cada una y después viene una cola de variantes con veintitantas. Esa forma, unos pocos valores frecuentes y una cola larga de valores raros, es la firma de un problema de escritura.

1.4 Lo que falta

```
# 1.4
# Los valores vacíos de mwh llegaron como nulos porque la columna es texto.
print("celdas vacías en mwh", sucia["mwh"].isna().sum())
print("tipo de la columna mwh", sucia["mwh"].dtype)
print("tipo de la columna fecha", sucia["fecha"].dtype)
```

```
celdas vacías en mwh 1846
tipo de la columna mwh object
tipo de la columna fecha object
```

1.846 celdas vacías en mwh, un 3 por ciento del archivo. Aparecen como nulos porque la columna es texto. Si el vacío hubiera venido como cero, no se verían.

OJO CON LO SUCIO QUE ESTÁ ESTE ARCHIVO

Los siete defectos son reales, pero venir todos juntos en el mismo CSV es un montaje del curso. En el trabajo aparecen de a uno y repartidos en meses.

Los tipos de defecto sí los vas a encontrar. La coma decimal y la fecha chilena con el día primero están en cualquier planilla exportada acá. Lo que no es realista es la densidad, cuarenta y nueve formas de escribir siete regiones es mucho más de lo que trae un archivo oficial.

Tampoco son reales las magnitudes. La demanda máxima del sistema de este lab es de 2.379,8 MWh por hora y la máxima real del Sistema Eléctrico Nacional en 2025 fue de 13.093,1 MWh por hora¹.

¹Coordinador Eléctrico Nacional, Reporte Energético SEN de enero de 2026, que informa la demanda máxima bruta

La solución del ejercicio

El ejercicio pedía ver las variantes mal escritas.

```
print(sucia["region"].value_counts().head(20))
```

De las 49 formas distintas, 42 son variantes con pocas filas. Las más frecuentes son biobío con 26 filas, los lagos con 21 y BIOBÍO con 21. Las siete correctas tienen más de 8.600 filas cada una.

Referencia rápida

Instrucción	Para qué sirve
<code>df.info()</code>	Tipos, conteo de no nulos y memoria, todo junto
<code>df.shape</code>	Filas y columnas
<code>df.head(n)</code>	Las primeras filas, para ver cómo llegó
<code>df["col"].nunique()</code>	Cuántos valores distintos hay
<code>df["col"].value_counts()</code>	Cuántas filas tiene cada valor
<code>df.isna().sum()</code>	Cuántos faltan, por columna
<code>!head -3 archivo.csv</code>	Ver el archivo crudo en Colab, sin que pandas lo interprete

Errores comunes

Lo que aparece en pantalla	Qué pasó
Una sola columna con todo el contenido adentro	El separador no es el que pandas supuso. Acá es punto y coma.
Todas las columnas salen como object	El archivo se leyó bien pero los tipos hay que arreglarlos. No es un error, es el punto de partida del bloque 2.
El conteo de filas no calza con lo esperado	No es un error de pandas, es un hallazgo. Hay filas de más o de menos.

Tres preguntas de repaso

1. Cuáles son las cuatro preguntas del diagnóstico

Cuántas filas tiene contra cuántas debería tener, de qué tipo es cada columna, cuántos valores distintos hay donde debería haber pocos y cuántos faltan.

2. Cómo se sabe cuántas filas debería tener un archivo

Con una cuenta de periodos por unidades. Acá, días por horas por regiones. Si esa cuenta no existe, hay que preguntársela a quien entrega el archivo.

3. Qué indica que una columna tenga muchos valores distintos con muy pocas filas cada uno

Casi siempre un problema de escritura, no un dato nuevo. Son variantes del mismo valor.

3. Bloque 2, tipos y conversiones

Qué se vio

Las etiquetas escritas a mano. Un archivo chileno trae el número con coma decimal y la fecha con el día primero, y pandas no entiende ninguno de los dos sin ayuda.

Lo peligroso es que no fallan. La columna de números llega como texto y todo sigue funcionando hasta que alguien la suma. La fecha llega como texto y se puede ordenar, solo que ordena mal.

El código, paso a paso

2.1 Sumar texto no da error

```
# 2.1
# Sumar una columna de texto no da error, concatena, que es peor.
print(sucia["mwh"].head(3).sum()[:40])
print()
print("no es un número, son tres textos pegados")
```

```
51,9549,0248,22
```

```
no es un número, son tres textos pegados
```

No hay error. Pandas pegó los tres textos uno detrás de otro, porque para él eso es sumar texto. Este es el peor tipo de error que existe, el que no se queja.

2.2 El arreglo en dos pasos

```
# 2.2
# str da acceso a las operaciones de texto sobre toda la columna de una vez.
limpia = sucia.copy()
limpia["mwh"] = limpia["mwh"].str.replace(",", ".", regex=False).astype(float)

print(limpia["mwh"].dtype)
print("suma real", round(limpia["mwh"].sum(), 1), "MWh")
```

```
float64
suma real 13560559.7 MWh
```

str da acceso a las operaciones de texto sobre toda la columna y astype convierte. El regex=False importa, porque sin él el punto y la coma se interpretan como expresión regular.

2.3 El arreglo en la lectura

```
# 2.3
# La forma corta, avisándole a pandas en la lectura en vez de arreglar después.
prueba = pd.read_csv("demanda_sucia.csv", sep=";", decimal=",")

print("tipo de mwh leyendo con decimal", prueba["mwh"].dtype)
```

```
tipo de mwh leyendo con decimal float64
```

La forma corta y la preferible. Se le avisa a la lectura que el decimal es coma y la columna llega numérica de entrada, sin un paso extra que alguien pueda olvidar al repetir el proceso.

2.4 La fecha, con el formato escrito a mano

```
# 2.4
# format le dice exactamente cómo está escrita la fecha en el archivo.
limpia["fecha"] = pd.to_datetime(limpia["fecha"], format="%d/%m/%Y")

print(limpia["fecha"].dtype)
print("primer día", limpia["fecha"].min().date(), " último día", limpia["fecha"].max().date())
```

```
datetime64[ns]
primer día 2024-01-01 último día 2024-12-31
```

El formato se escribe a mano aunque sea más largo. Si una fila no calza con ese formato, la línea falla, y fallar es lo que uno quiere. Si se deja que pandas adivine, el 3 de junio se puede leer como 6 de marzo, sin aviso.

2.6 Las regiones, normalizadas

```
# 2.6
# strip saca los espacios de los extremos y lower pasa todo a minúsculas.
limpia["region"] = limpia["region"].str.strip().str.lower()

print("regiones distintas después de normalizar", limpia["region"].nunique())
print(sorted(limpia["region"].unique()))
```

```
regiones distintas después de normalizar 7
['antofagasta', 'atacama', 'biobío', 'coquimbo', 'los lagos', 'metropolitana', 'valparaíso']
```

strip saca los espacios de los extremos y lower pasa todo a minúsculas. Con esas dos operaciones las 49 variantes se convierten en 7, sin escribirlas una por una.

2.7 De vuelta a la forma oficial

```
# 2.7
# Y un diccionario devuelve cada una a su forma oficial.
CANONICAS = {"antofagasta": "Antofagasta", "atacama": "Atacama", "coquimbo": "Coquimbo",
             "valparaíso": "Valparaíso", "metropolitana": "Metropolitana",
             "biobío": "Biobío", "los lagos": "Los Lagos"}
limpia["region"] = limpia["region"].map(CANONICAS)

print("regiones", sorted(limpia["region"].unique()))
print("sin mapear", limpia["region"].isna().sum())
```

```
regiones ['Antofagasta', 'Atacama', 'Biobío', 'Coquimbo', 'Los Lagos', 'Metropolitana', 'Valparaíso']
sin mapear 0
```

Un diccionario las devuelve a como se escriben en un informe. La línea que cuenta los no mapeados es la comprobación, si alguna variante se escapa, queda como nulo y conviene enterarse ahora.

2.8 La columna como categoría

```
# 2.8
# category guarda cada texto una sola vez y deja números en las filas.
antes = limpia["region"].memory_usage(deep=True) / 1024
limpia["region"] = limpia["region"].astype("category")
despues = limpia["region"].memory_usage(deep=True) / 1024

print(f"memoria de la columna region, antes {antes:.0f} KB, después {despues:.0f} KB")
```

```
memoria de la columna region, antes 3939 KB, después 61 KB
```

category guarda cada texto una vez y deja números en las filas. La memoria de la columna baja de 3.939 KB a 61 KB. Sirve con pocas categorías repetidas muchas veces, y no sirve con identificadores casi únicos.

La solución del ejercicio

El ejercicio pedía resolver las dos conversiones en la propia lectura.

```
otra = pd.read_csv("demanda_sucia.csv", sep=";", decimal=".",
                  parse_dates=["fecha"], dayfirst=True)

print(otra.dtypes)
```

Con eso fecha llega como datetime64 y mwh como float64. La región sigue siendo texto sin normalizar, porque eso no se puede resolver en la lectura.

Referencia rápida

Instrucción	Para qué sirve
<code>sep=";"</code>	Separador de columnas distinto de la coma
<code>decimal=","</code>	Número con coma decimal
<code>encoding="utf-8-sig"</code>	Archivo con marca de orden de bytes al inicio
<code>parse_dates=["fecha"], dayfirst=True</code>	Fecha chilena en la lectura
<code>pd.to_datetime(col, format="%d/%m/%Y")</code>	Conversión estricta, falla si no calza
<code>col.str.replace(",", ".", regex=False)</code>	Reemplazo de texto en toda la columna
<code>col.astype(float)</code>	Convertir a número
<code>col.str.strip().str.lower()</code>	Normalizar texto
<code>col.map(diccionario)</code>	Traducir a la forma oficial
<code>col.astype("category")</code>	Ahorrar memoria con pocas categorías

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>ValueError: could not convert string to float</code>	Quedó algún carácter que no es número, por ejemplo el separador de miles o un espacio.
<code>ValueError: time data "13/06/2024" doesn't match format</code>	El formato que escribiste no calza con el archivo. Acá es buena noticia, te está avisando.
La columna nueva queda llena de NaN después de map	Hay valores que no están en el diccionario. Míralos con <code>value_counts</code> antes de mapear.
<code>AttributeError: Can only use .str accessor with string values</code>	La columna ya no es texto, probablemente la convertiste antes.

Tres preguntas de repaso

1. Por qué se escribe el formato de la fecha a mano

Porque si una fila no calza, la conversión falla y te avisa. Si pandas adivina, puede leer el día como mes y cambiar el dato sin decir nada.

2. Qué hacen `strip` y `lower` juntos

Sacan los espacios de los extremos y pasan todo a minúsculas, con lo que las variantes de escritura de una misma categoría se juntan en una sola.

3. Cuándo conviene el tipo `category`

Cuando hay pocos valores distintos repetidos muchas veces. Con identificadores casi únicos ocupa más que el texto normal.

4. Bloque 3, filas repetidas

Qué se vio

El bulto que contaron dos cuadrillas. Hay dos casos distintos y la función que los busca es la misma, pero lo que se hace después no lo es.

CASO 1, LA FILA ENTERA REPETIDA

fecha	hora	region	mwh
01-01	18	Antofagasta	74,81
01-01	18	Antofagasta	74,81

Idénticas en todo. Son las 20 que plantamos. `drop_duplicates` deja una.

CASO 2, LA CLAVE REPETIDA

fecha	hora	region	mwh
01-01	18	Antofagasta	74,81
01-01	18	Antofagasta	92,10

Misma hora y región, distinto valor. Acá hay que decidir cuál vale, y eso no lo resuelve ninguna función.

Los dos se buscan con `uplicated`. El primero sin argumentos, el segundo con `subset` y la lista de columnas que forman la clave. En este archivo el segundo caso no aparece, y por eso conviene comprobarlo igual, porque en los datos de verdad es el que más daño hace.

El código, paso a paso

3.1 Cuántas filas están repetidas enteras

```
# 3.1
# duplicated marca cada fila que ya apareció antes, idéntica en todas las columnas.
print("filas duplicadas exactas", limpia.duplicated().sum())
```

```
filas duplicadas exactas 20
```

Veinte, el mismo número que sobraba en el bloque 1. Ahí está la explicación de esa diferencia.

3.2 Verlas en pareja

```
# 3.2
# keep=False marca las dos copias, no solo la segunda, para verlas en pareja.
repetidas = limpia[limpia.duplicated(keep=False)].sort_values(["fecha", "hora", "region"])

print(repetidas.shape[0], "filas involucradas")
repetidas.head(4)
```

```
40 filas involucradas
      fecha  hora  region  mwh
18  2024-01-01   18  Antofagasta  74.81
19  2024-01-01   18  Antofagasta  74.81
254 2024-01-11   13  Antofagasta  71.42
255 2024-01-11   13  Antofagasta  71.42
```

Por defecto `duplicated` marca solo la segunda aparición. Con `keep=False` marca las dos, que es lo que sirve para mirarlás juntas.

3.3 Dejar una sola

```
# 3.3
# drop_duplicates conserva la primera aparición y descarta las repeticiones.
antes = len(limpia)
limpia = limpia.drop_duplicates()

print("antes", antes, " después", len(limpia), " se fueron", antes - len(limpia))
```

```
antes 61508  después 61488  se fueron 20
```

`drop_duplicates` conserva la primera aparición. La tabla queda en 61.488 filas, exactamente lo que decía la cuenta teórica. Ese calce es la comprobación.

3.4 La clave lógica

```
# 3.4
# subset limita la comparación a las columnas que forman la clave lógica.
CLAVE = ["fecha", "hora", "region"]

print("claves repetidas", limpia.duplicated(subset=CLAVE).sum())
print("filas", len(limpia), "y esperadas", 366 * 24 * 7)
```

```
claves repetidas 0
filas 61488 y esperadas 61488
```

Esta es la comprobación que de verdad importa. Puede no haber filas idénticas y aun así tener la misma hora de la misma región dos veces, con valores distintos. Acá da cero, y conviene escribirla igual, siempre.

La solución del ejercicio

El ejercicio pedía comprobar que las siete regiones tuvieran la misma cantidad de filas.

```
print(limpia.duplicated(subset=CLAVE).sum(), "claves repetidas")
print(limpia["region"].value_counts())
```

Las siete regiones tienen 8.784 filas cada una, que son las horas del año 2024. Si alguna diera distinto, habría filas que faltan o que sobran en esa región.

Referencia rápida

Instrucción	Para qué sirve
<code>df.duplicated()</code>	Marca filas repetidas enteras
<code>df.duplicated(keep=False)</code>	Marca las dos copias, para verlas en pareja
<code>df.duplicated(subset=CLAVE)</code>	Marca claves repetidas aunque el valor difiera
<code>df.drop_duplicates()</code>	Deja una sola copia, la primera
<code>df.drop_duplicates(keep="last")</code>	Deja la última, útil con archivos corregidos
<code>df.reset_index(drop=True)</code>	Renumeración el índice después de borrar

Tres preguntas de repaso

1. Qué diferencia hay entre fila repetida y clave repetida

La fila repetida es idéntica en todas las columnas y se descarta sola. La clave repetida tiene la misma identificación con valores distintos, y ahí hay que decidir cuál vale.

2. Cuál copia conserva `drop_duplicates`

La primera. Con `keep="last"` conserva la última, que es lo razonable cuando el archivo trae correcciones al final.

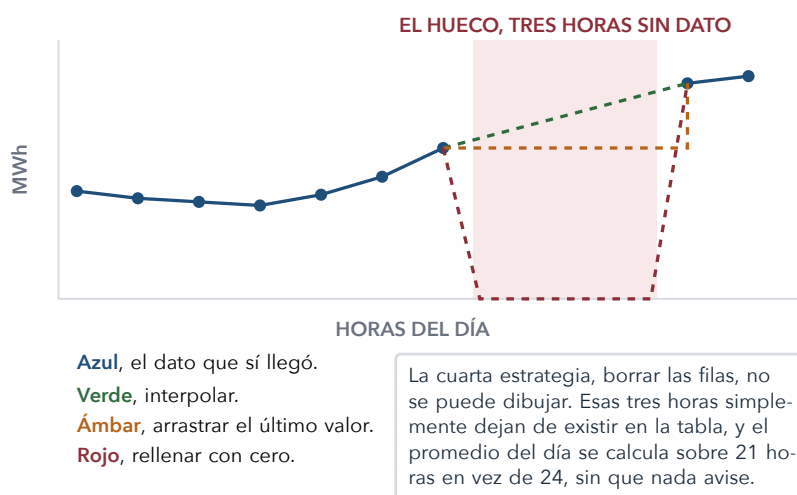
3. Por qué escribir la comprobación de la clave si dio cero

Porque el cero es el resultado de hoy, no una garantía. En el archivo del mes que viene puede no serlo, y la comprobación escrita lo detecta sola.

5. Bloque 4, datos que faltan

Qué se vio

Las cajas que no llegaron. Este es el bloque donde no hay respuesta correcta. Cuatro estrategias razonables dan cuatro resultados distintos sobre los mismos datos.



El código, paso a paso

4.1 Cuántos faltan

```
# 4.1
# isna marca los faltantes y sum los cuenta por columna.
# Son 1.844 y no los 1.846 del diagnóstico, porque 2 de las 20 filas duplicadas
# que se sacaron venían vacías.
print(limpia.isna().sum())
print()
print("porcentaje de mwh sin dato", round(100 * limpia["mwh"].isna().mean(), 2))
```

```
fecha      0
hora       0
region     0
mwh      1844
dtype: int64

porcentaje de mwh sin dato 3.0
```

1.844 valores sin dato, un 3 por ciento de la tabla. Son dos menos que los 1.846 del bloque 1, porque dos estaban en las filas duplicadas que ya se borraron.

4.2 Dónde están

```
# 4.2
# Para todo lo que viene, el orden importa.
limpia = limpia.sort_values(CLAVE).reset_index(drop=True)

# Los nulos no están repartidos parejo, se concentran en bloques.
print(limpia.groupby("region", observed=True)["mwh"].apply(lambda s: s.isna().sum()))
```

```
region
Antofagasta    222
Atacama        272
Biobío         322
Coquimbo       262
Los Lagos      241
Metropolitana  277
Valparaíso     248
Name: mwh, dtype: int64
```

No están repartidos parejo entre regiones. Y la tabla se ordena por la clave, porque cualquier método que use el vecino de al lado necesita que el vecino sea el correcto.

4.3 Cómo se ve un hueco de verdad

```
# 4.3
# Un día concreto de Valparaíso, para ver cómo se ve un hueco de verdad.
hueco = limpia[(limpia["region"] == "Valparaíso")
               & (limpia["fecha"] == "2024-09-16")]

print(hueco[["hora", "mwh"]].to_string(index=False))
```

```
hora    mwh
0  177.03
1    NaN
2    NaN
3  165.98
4  164.67
5  179.25
6  191.42
7  222.46
8    NaN
9    NaN
10 247.39
11 259.55
... [y así hasta la hora 23]
```

Un día de Valparaíso, con bloques de dos horas sin dato. No es ruido aleatorio, son cortes. Eso importa, porque borrar las filas incompletas elimina pedazos del día y no filas sueltas.

4.4 Estrategia 1, borrar

```
# 4.4
# Estrategia 1, borrar las filas incompletas.
print("antes ", len(limpia))
print("después ", len(limpia.dropna(subset=["mwh"])))
print("se pierden", limpia["mwh"].isna().sum(), "filas y con ellas sus horas")
```

```
antes      61488
después    59644
se pierden 1844 filas y con ellas sus horas
```

Se van 1.844 filas y con ellas horas completas. El promedio que se calcule después ya no es el del día, es el de las horas que sobrevivieron. Borrar es legítimo cuando falta poco y falta al azar, y acá no se cumple ninguna de las dos.

4.5 Estrategia 2, rellenar con cero

```
# 4.5
# Estrategia 2, rellenar con un valor fijo, casi siempre una mala idea.
con_cero = limpia["mwh"].fillna(0)

print("promedio real      ", round(limpia["mwh"].mean(), 1))
print("promedio con ceros", round(con_cero.mean(), 1))
```

```
promedio real      227.3
promedio con ceros 220.5
```

El promedio real es 227,3 y con ceros baja a 220,5. Casi siete MWh de diferencia por una decisión de relleno. En demanda eléctrica un cero no es un dato faltante, es afirmar que nadie consumió nada esa hora.

4.6 Estrategias 3 y 4, arrastrar e interpolar

```
# 4.6
# Estrategia 3, arrastrar el último valor conocido, dentro de cada región.
adelante = limpia.groupby("region", observed=True)["mwh"].ffill()

# Estrategia 4, interpolar, o sea trazar una recta entre los dos extremos.
interpolada = limpia.groupby("region", observed=True)["mwh"].transform(
    lambda s: s.interpolate())

print("nulos que quedan, arrastrando ", adelante.isna().sum())
print("nulos que quedan, interpolando", interpolada.isna().sum())
```

```
nulos que quedan, arrastrando  0
nulos que quedan, interpolando 0
```

Las dos rellenan todo. Fíjate en que van agrupadas por región, porque arrastrar el último valor conocido de otra región no tiene ningún sentido. Ese groupby es el error más común del bloque.

4.7 Las cuatro, lado a lado

```
# 4.7
# Las cuatro estrategias sobre el mismo hueco, una al lado de la otra.
comparacion = pd.DataFrame({
    "hora": limpia.loc[hueco.index, "hora"],
    "original": limpia.loc[hueco.index, "mwh"],
    "con_cero": con_cero.loc[hueco.index],
    "arrastrado": adelante.loc[hueco.index],
    "interpolado": interpolada.loc[hueco.index],
}).round(1)

print(comparacion.head(14).to_string(index=False))
```

hora	original	con_cero	arrastrado	interpolado
0	177.0	177.0	177.0	177.0
1	NaN	0.0	177.0	173.3
2	NaN	0.0	177.0	169.7
3	166.0	166.0	166.0	166.0
4	164.7	164.7	164.7	164.7
5	179.2	179.2	179.2	179.2
6	191.4	191.4	191.4	191.4
7	222.5	222.5	222.5	222.5
8	NaN	0.0	222.5	230.8
9	NaN	0.0	222.5	239.1
10	247.4	247.4	247.4	247.4
11	259.6	259.6	259.6	259.6
... [y sigue hasta la hora 23]				

Acá está el bloque entero en una tabla. En la hora 1 el original no existe, el cero dice cero, el arrastre dice 177,0 y la interpolación dice 173,3. Tres respuestas distintas para la misma hora, y ninguna es el dato real.

4.8 La decisión, y la huella

```
# 4.8
# Guardamos primero qué filas venían vacías, para poder auditarlas al final.
limpia["fue_nulo"] = limpia["mwh"].isna()
limpia["mwh"] = interpolada

print("filas marcadas como rellenadas", limpia["fue_nulo"].sum())
print("nulos que quedan", limpia["mwh"].isna().sum())
```

```
filas marcadas como rellenadas 1844
nulos que quedan 0
```

La línea importante es la primera. Antes de rellenar nada, se guarda qué filas venían vacías. Esa columna es lo que después permite decir cuánto del resultado es dato medido y cuánto es estimación nuestra. En el bloque 5 aparece una segunda marca, `fue_corregido`, para los valores medidos que se cambian por imposibles.

La solución del ejercicio

El ejercicio pedía rellenar con el promedio de la misma región y la misma hora del día.

```
por_region_hora = limpia.groupby(["region", "hora"], observed=True)["mwh"].transform("mean")
limpia["mwh"] = limpia["mwh"].fillna(por_region_hora)
```

LO INTERESANTE ES QUE ESA ESTRATEGIA PIERDE

Suena mejor que interpolar, porque respeta la forma del día. Medida contra el dato real en el bloque 7, sobre las mismas 1.844 filas que venían vacías, da un error medio de 27,01 MWh, mientras la interpolación da 5,40.

La razón es que el promedio de una hora mezcla los doce meses del año, y la demanda de julio no se parece a la de enero. La interpolación, en cambio, usa las horas vecinas del mismo día, que sí son comparables.

Es el mejor ejemplo del módulo de por qué estas decisiones se miden y no se suponen.

Referencia rápida

Instrucción	Para qué sirve
<code>df.isna().sum()</code>	Cuántos faltan por columna
<code>df["col"].isna().mean()</code>	Qué proporción falta
<code>df.dropna(subset=["col"])</code>	Borrar las filas incompletas
<code>df["col"].fillna(0)</code>	Rellenar con un valor fijo
<code>df.groupby(g) ["col"].ffill()</code>	Arrastrar el último valor conocido
<code>.transform(lambda s: s.interpolate())</code>	Interpolar dentro de cada grupo
<code>df["fue_nulo"] = df["col"].isna()</code>	Dejar registro antes de rellenar
<code>df.sort_values(CLAVE)</code>	Ordenar, obligatorio antes de rellenar por vecino

Errores comunes

Lo que aparece en pantalla	Qué pasó
El relleno mezcla valores de otra región	Falta agrupar por región antes de <code>ffill</code> o de <code>interpolate</code> .
Quedan nulos después de interpolar	Son nulos al principio de la serie, que no tienen vecino anterior. Se resuelven con <code>bfill</code> o se dejan.
El promedio cambió y no sé por qué	Rellenaste con un valor fijo. Cualquier relleno cambia los estadísticos, por eso se declara.

Tres preguntas de repaso

1. Por qué borrar las filas incompletas es peligroso acá

Porque los nulos vienen en bloques, así que borrar elimina periodos completos y no una muestra al azar.

2. Por qué rellenar con cero es casi siempre mala idea en demanda

Porque un cero afirma que no hubo consumo, que es un dato fuerte y falso. Además arrastra el promedio hacia abajo.

3.Cuál es la línea más importante del bloque

La que crea fue_nulo, antes de rellenar. Sin ella no se puede saber después qué filas son medición y cuáles son estimación.

6. Bloque 5, valores imposibles

Qué se vio

El bulto que pesa menos que nada. Hay dos clases de valor raro que conviene no confundir. El extremo, que es alto pero posible, y el imposible, que viola una regla del mundo.

Se probaron cuatro formas de detectarlos y el resultado es incómodo, el método más sofisticado es el único que no encuentra los valores imposibles.

método	filas que marca	negativos que pilla, de 15
regla de negocio, mwh < 0	15	15 de 15
rango intercuartílico	59	15 de 15
z-score	59	15 de 15
Isolation Forest	308	0 de 15

El método más sofisticado de los cuatro es el único que no encuentra los valores imposibles. Marca 308 filas, todas picos altos, y ni uno solo de los 15 negativos.

La regla de negocio, una línea que dice que la demanda no puede ser negativa, es la que tiene mejor rendimiento de las cuatro. Marca exactamente lo que está mal y nada más.

El código, paso a paso

5.1 La regla de negocio

```
# 5.1
# La regla de negocio más simple que existe, y la más efectiva.
negativos = limpia[limpia["mwh"] < 0]

print("filas con demanda negativa", len(negativos))
negativos[["fecha", "hora", "region", "mwh"]].head(3)
```

```
filas con demanda negativa 15
      fecha  hora  region  mwh
52  2024-01-01    7  Coquimbo -64.74
6875 2024-02-10   22  Atacama  -4.36
8251 2024-02-19    2  Metropolitana -493.42
```

Quince filas con demanda negativa. Una línea de código y cero estadística. La demanda eléctrica no puede ser negativa, y eso no lo sabe el algoritmo, lo sabes tú.

5.2 La forma de la variable

```
# 5.2
# describe da la forma de la variable en ocho números.
limpia["mwh"].describe().round(1)
```

```
count    61488.0
mean      227.5
std       302.5
min       -493.4
25%        73.1
50%       111.5
75%       210.9
max      11195.0
Name: mwh, dtype: float64
```

El mínimo es negativo y el máximo es 11.195 cuando la mediana es 111,5. Cuando el máximo es cien veces la mediana, casi siempre hay un problema de datos y no un evento extraordinario.

5.3 y 5.4 Los dos métodos clásicos

```
# 5.3
# Método 1, rango intercuartílico, calculado dentro de cada región.
q1 = limpia.groupby("region", observed=True)["mwh"].transform(lambda s: s.quantile(0.25))
q3 = limpia.groupby("region", observed=True)["mwh"].transform(lambda s: s.quantile(0.75))
rango = q3 - q1
fuera_iqr = (limpia["mwh"] < q1 - 1.5 * rango) | (limpia["mwh"] > q3 + 1.5 * rango)

print("marcadas por rango intercuartílico", fuera_iqr.sum())
```

```
marcadas por rango intercuartílico 59
```

```
# 5.4
# Método 2, z-score, que mide a cuántas desviaciones está cada valor de su promedio.
media = limpia.groupby("region", observed=True)["mwh"].transform("mean")
desv = limpia.groupby("region", observed=True)["mwh"].transform("std")
z = (limpia["mwh"] - media) / desv
fuera_z = z.abs() > 3

print("marcadas por z-score", fuera_z.sum())
```

```
marcadas por z-score 59
```

El rango intercuartílico marca lo que está a más de una vez y media el rango entre cuartiles. El z-score marca lo que está a más de tres desviaciones del promedio. Los dos calculados dentro de cada región, porque la Metropolitana y Atacama no se comparan con el mismo umbral.

5.5 Isolation Forest

```
# 5.5
# Método 3, Isolation Forest, que no mira una columna sino la combinación.
from sklearn.ensemble import IsolationForest

X = limpia[["hora", "mwh"]]
modelo = IsolationForest(contamination=0.005, random_state=2026)
fuera_bosque = modelo.fit_predict(X) == -1

print("marcadas por Isolation Forest", fuera_bosque.sum())
```

```
marcadas por Isolation Forest 308
```

Esto es aprendizaje automático y son tres líneas. No se implementa nada, se llama a una biblioteca. Busca puntos que se aíslan fácil del resto mirando la combinación de hora y megawatt hora. contamination es la proporción de filas que uno espera que sean anómalas, o sea un supuesto de quien lo usa.

5.6 Los cuatro, comparados

```
# 5.6
# Los tres métodos, lado a lado, y lo que más importa, cuántos negativos pilla cada uno.
es_negativo = limpia["mwh"] < 0

for nombre, marca in [("rango intercuartílico", fuera_iqr), ("z-score", fuera_z),
                      ("Isolation Forest", fuera_bosque)]:
    print(f"{nombre:24} marca {marca.sum():5} filas, "
          f"y pilla {(marca & es_negativo).sum():2} de los {es_negativo.sum()} negativos")
```

rango intercuartílico	marca	59 filas, y pilla 15 de los 15 negativos
z-score	marca	59 filas, y pilla 15 de los 15 negativos
Isolation Forest	marca	308 filas, y pilla 0 de los 15 negativos

Acá está el punto. Los dos métodos clásicos marcan 59 filas y pillan los 15 negativos. Isolation Forest marca 308 y no pilla ninguno. No está mal programado, le hicimos otra pregunta, y en el espacio de hora contra megawatt hora los negativos no son raros porque hay regiones con valores chicos.

5.7 y 5.8 Qué hacer con lo que encontramos

```
# 5.7
import numpy as np

# Convertimos a nulo lo que los dos métodos clásicos marcaron juntos, y los negativos.
sospechosas = (fuera_iqr & fuera_z) | es_negativo
# Antes de tocarlas se marcan, igual que los nulos. Las que venían vacías ya tienen
# su marca en fue_nulo, así que acá se marcan solo los valores medidos.
limpia["fue_corregido"] = sospechosas & ~limpia["fue_nulo"]
limpia["mwh"] = np.where(sospechosas, np.nan, limpia["mwh"])

print("filas convertidas a nulo", sospechosas.sum())
print("valores medidos marcados como corregidos", limpia["fue_corregido"].sum())
print("nulos que quedan", limpia["mwh"].isna().sum())
```

```
filas convertidas a nulo 59
valores medidos marcados como corregidos 55
nulos que quedan 59
```

```
# 5.8
# Y se rellenan igual que antes, interpolando dentro de cada región.
limpia["mwh"] = limpia.groupby("region", observed=True)["mwh"].transform(
    lambda s: s.interpolate())

limpia["mwh"].describe().round(1)
```

```
count    61488.0
mean      226.1
std       285.3
min        19.2
25%        73.1
50%       111.5
75%       210.7
max       1352.1
Name: mwh, dtype: float64
```

Se convierte a nulo lo que los dos métodos clásicos marcaron juntos, más los negativos, y se rellena igual que en el bloque anterior. El mínimo deja de ser negativo y el máximo baja de 11.195 a 1.352,1. Fíjate en que no se borran filas, así el largo de la tabla no cambia.

Antes de convertir, se marca. Es la misma regla del bloque 4, primero la marca y después el cambio. De las 59 filas, 4 venían vacías y ya tenían su marca en fue_nulo, así que fue_corregido marca las 55 que eran valores medidos. Sin esta segunda marca, en la auditoría del bloque 7 esas 55 filas quedarían mezcladas con las que nunca se tocaron.

La solución del ejercicio

El ejercicio pedía mover contamination.

```
for c in (0.001, 0.005, 0.01):
    modelo = IsolationForest(contamination=c, random_state=2026)
    marcadas = modelo.fit_predict(limpia[["hora", "mwh"]]) == -1
    print(c, marcadas.sum(), "filas")
```

Marca 58, 308 y 614 filas respectivamente. La relación es casi exacta con el porcentaje de la tabla, así que el número de anomalías no lo descubre el algoritmo, se lo dices tú.

Referencia rápida

Instrucción	Para qué sirve
<code>df[df["col"] < 0]</code>	La regla de negocio, lo imposible
<code>df["col"].describe()</code>	La forma de la variable en ocho números
<code>.transform(lambda s: s.quantile(0.25))</code>	Cuartiles por grupo
<code>(col - media) / desv</code>	z-score, distancia en desviaciones
<code>IsolationForest(contamination=c)</code>	Anomalías por combinación de columnas
<code>np.where(cond, np.nan, col)</code>	Convertir a nulo en vez de borrar la fila

Tres preguntas de repaso

1. Cuál es la diferencia entre valor extremo y valor imposible

El extremo es alto pero puede ocurrir. El imposible viola una regla física o de negocio, como una demanda negativa, y se detecta con una condición, no con estadística.

2. Por qué los umbrales se calculan por región

Porque la Metropolitana tiene una demanda diez veces mayor que Atacama. Un umbral global marcaría como anómala la operación normal de la región grande.

3. Qué es contamination

La proporción de filas que el algoritmo va a marcar como anómalas. La fija quien lo usa, no el algoritmo.

7. Bloque 6, describir lo que quedó

Qué se vio

Describir la carga en cinco números. El promedio solo es una mala descripción de una variable que tiene forma de día, y la mediana al lado dice si el promedio sirve.

El código, paso a paso

6.1 Describe con percentiles

```
# 6.1
# describe acepta los percentiles que uno quiera, no solo los cuartiles.
limpia["mwh"].describe(percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]).round(1)
```

```
count    61488.0
mean      226.1
std       285.3
min       19.2
5%        31.7
25%       73.1
50%      111.5
75%      210.7
95%      961.2
max      1352.1
Name: mwh, dtype: float64
```

El percentil 5 es 31,7 y el 95 es 961,2, o sea treinta veces más. Esos dos describen el rango de trabajo mejor que el mínimo y el máximo, que siempre son casos raros.

6.2 Promedio contra mediana

```
# 6.2
# La asimetría dice hacia qué lado se estira la cola de la distribución.
print("asimetría", round(limpia["mwh"].skew(), 3))
print("mediana ", round(limpia["mwh"].median(), 1))
print("promedio ", round(limpia["mwh"].mean(), 1))
```

```
asimetría 2.101
mediana   111.5
promedio  226.1
```

Asimetría 2,101, positiva, o sea cola hacia la derecha. El promedio es 226,1 y la mediana 111,5, así que la mitad de las horas está por debajo de la mitad del promedio.

OJO CON ESTA DISTANCIA ENTRE PROMEDIO Y MEDIANA

Esa distancia no viene de valores extremos, que ya se corrigieron en el bloque 5. Viene de mezclar en una sola columna regiones de tamaño muy distinto, con la Metropolitana consumiendo varias veces más que las demás. En la tabla por región de la celda siguiente, dentro de cada región el promedio y la mediana están cerca.

6.4 El mismo describe, por región

```
# 6.4
# describe también funciona después de un groupby, una fila por grupo.
limpia.groupby("region", observed=True)["mwh"].describe().round(1)
```

region	count	mean	std	min	25%	50%	75%	max
Antofagasta	8784.0	76.6	16.2	41.9	64.8	75.5	89.3	116.3
Atacama	8784.0	35.0	7.4	19.2	29.7	34.5	40.9	53.4
Biobio	8784.0	185.9	39.4	102.2	157.4	183.1	216.7	282.3
Coquimbo	8784.0	93.0	19.7	51.4	78.8	91.6	108.6	141.9
Los Lagos	8784.0	98.4	20.9	53.6	83.5	97.1	115.0	149.6
Metropolitana	8784.0	885.7	187.6	484.5	750.9	873.4	1033.8	1352.1
Valparaíso	8784.0	207.9	44.0	114.8	176.4	204.6	242.3	315.4

Y acá está la explicación. La asimetría no era de la demanda, era de mezclar siete regiones con escalas muy distintas. Dentro de cada una, promedio y mediana se parecen mucho más.

6.5 y 6.6 El perfil horario y las franjas

```
# 6.5
# Perfil horario promedio de cada región, una columna por región.
perfil = limpia.pivot_table(index="hora", columns="region", values="mwh",
                             aggfunc="mean", observed=True)

perfil.round(1).head(6)
```

```
region  Antofagasta  Atacama  Biobío  Coquimbo  Los Lagos  Metropolitana  \
hora
0          61.5      28.1    149.5      74.8      79.2          712.3
1          58.1      26.6    141.2      70.7      74.8          673.0
2          56.4      25.7    136.8      68.5      72.6          652.6
3          55.5      25.4    135.0      67.4      71.5          642.5
4          56.3      25.8    137.0      68.5      72.7          652.7
5          59.8      27.4    145.8      72.7      77.0          693.1

region  Valparaíso
hora
0          167.1
1          157.9
2          153.4
3          150.9
4          153.3
5          162.7
```

```
# 6.6
# Una franja horaria con pd.cut, como en el Módulo 2, y la demanda media de cada una.
limpia["franja"] = pd.cut(limpia["hora"], bins=[-1, 5, 11, 17, 23],
                           labels=["madrugada", "mañana", "tarde", "noche"])

pd.crosstab(limpia["region"], limpia["franja"],
             values=limpia["mwh"], aggfunc="mean").round(1)
```

```
franja      madrugada  mañana  tarde  noche
region
Antofagasta      57.9     79.9   84.5   83.9
Atacama           26.5     36.5   38.6   38.4
Biobío           140.9    193.8  205.1  203.7
Coquimbo          70.4     97.0  102.6  101.8
Los Lagos         74.6    102.7  108.6  107.9
Metropolitana    671.0    923.8  977.5  970.3
Valparaíso       157.5    216.8  229.4  227.6
```

La pivot del Módulo 2 sobre la tabla limpia, y después las horas agrupadas en cuatro franjas con `pd.cut`. La tabla por franja es la que se pega en un informe.

UN ARTEFACTO DE LOS DATOS DEL CURSO

Las siete regiones tienen exactamente el mismo perfil horario, escalado por población, y por eso la correlación entre ellas da 1,000.

En el sistema real no es así. En el Norte Grande el consumo está dominado por la gran minería y la curva es casi plana, mientras en la zona centro predomina el consumo de clientes de distribución y la curva tiene mucha más forma.

La solución del ejercicio

El ejercicio pedía el percentil 95 por región.

```
print(limpia.groupby("region", observed=True)["mwh"].quantile(0.95).round(1))
```

Da 103,0 en Antofagasta, 47,0 en Atacama, 250,1 en Biobío, 124,9 en Coquimbo, 132,2 en Los Lagos, 1.190,4 en la Metropolitana y 279,2 en Valparaíso. Comparado con el promedio de cada región, la diferencia es lo que hay que tener en cuenta al dimensionar cualquier cosa.

Referencia rápida

Instrucción	Para qué sirve
<code>df["col"].describe(percentiles=)</code>	Los percentiles que uno quiera
<code>df["col"].skew()</code>	Asimetría, hacia qué lado se estira la cola
<code>df["col"].median()</code>	La mediana, al lado del promedio
<code>df.groupby(g)["col"].describe()</code>	El mismo resumen por grupo
<code>df["col"].quantile(0.95)</code>	El percentil alto, mejor que el máximo
<code>pd.crosstab(a, b, values=, aggfunc=)</code>	Tabla cruzada con un cálculo

Tres preguntas de repaso

1. Cuándo el promedio describe bien

Cuando se parece a la mediana. Si están lejos, hay cola larga y el promedio está tirado hacia el lado de la cola.

2. Qué significa una asimetría positiva

Que la cola se estira hacia la derecha, o sea hay valores altos que arrastran el promedio hacia arriba.

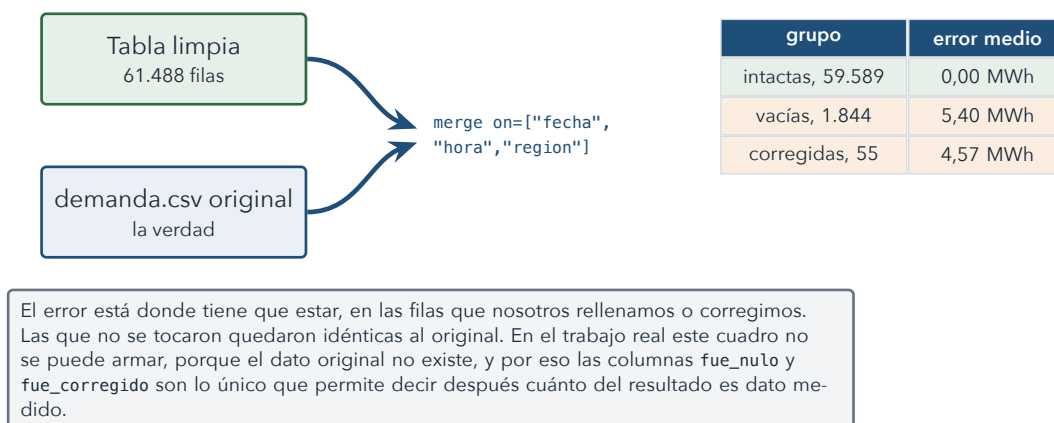
3. Por qué el describe por región cambia la conclusión

Porque la asimetría venía de mezclar regiones de escalas distintas, no de la forma de la demanda de cada una.

8. Bloque 7, contra la verdad

Qué se vio

El manifiesto original. Acá se hace algo que en el trabajo real no se puede, comparar la tabla limpia contra el dato verdadero, para entender qué significa la marca que sí vas a tener.



El código, paso a paso

7.1 El cruce contra el original

```
# 7.1
# Este lab tiene una ventaja que la vida real no da, el dato original existe.
verdad = pd.read_csv("demanda.csv", parse_dates=["fecha"])

auditoria = pd.merge(limpia, verdad, on=CLAVE, how="inner", suffixes=("_limpia", "_real"))
print(auditoria.shape[0], "filas comparadas")
```

61488 filas comparadas

El archivo sucio se fabricó a partir de uno limpio, así que se puede comparar. El cruce es el mismo del Módulo 2, por las tres columnas de la clave, y devuelve las 61.488 filas.

7.2 Cuánto nos equivocamos

```
# 7.2
# Error absoluto en MWh y error relativo en porcentaje, fila por fila.
auditoria["error"] = (auditoria["mwh_limpia"] - auditoria["mwh_real"]).abs()
auditoria["error_pct"] = 100 * auditoria["error"] / auditoria["mwh_real"]

print("error promedio   ", round(auditoria["error"].mean(), 2), "MWh")
print("error mediano    ", round(auditoria["error"].median(), 2), "MWh")
print("filas idénticas   ", (auditoria["error"] < 0.01).sum(), "de", len(auditoria))
```

error promedio 0.17 MWh
error mediano 0.0 MWh
filas idénticas 59595 de 61488

Error promedio de 0,17 MWh sobre una demanda promedio de 226. Y 59.595 filas de 61.488 quedaron idénticas al original. El error mediano es cero, porque a la mayoría de las filas no se les hizo nada.

7.3 Dónde está el error

```
# 7.3
# El error se concentra justo donde nosotros intervenimos. Tres grupos, según la marca.
auditoria["grupo"] = np.select([auditoria["fue_nulo"], auditoria["fue_corregido"]],
                              ["nulo original", "valor corregido"], default="fila intacta")

print(auditoria.groupby("grupo")["error"].agg(["count", "mean", "max"]).round(2))
```

	count	mean	max
grupo			
fila intacta	59589	0.00	0.00
nulo original	1844	5.40	65.97
valor corregido	55	4.57	47.97

Acá está la conclusión del módulo. Las dos marcas parten la tabla en tres grupos. Las 59.589 filas que no se tocaron tienen error cero, exacto. Las 1.844 que venían vacías tienen un error promedio de 5,40 MWh, y las 55 que se corrigieron por imposibles, de 4,57, con un máximo de 47,97. Sin las columnas fue_nulo y fue_corregido este cuadro no se puede hacer, y las filas corregidas quedarían escondidas entre las intactas.

7.4 Las dos estrategias, sobre las mismas filas

```
# 7.4
# La pregunta del Tu turno del bloque 4. ¿Rellenar con el promedio de la región y la
# hora es mejor que interpolar? Las dos, sobre las mismas filas, las que venían vacías.
# El promedio se calcula solo con las filas intactas, que son dato medido.
vacias = auditoria["fue_nulo"]
medido = auditoria["mwh_limpia"].where(auditoria["grupo"] == "fila intacta")
promedio_rh = medido.groupby([auditoria["region"], auditoria["hora"]],
                             observed=True).transform("mean")

estrategias = pd.DataFrame({
    "interpolar": auditoria.loc[vacias, "error"],
    "promedio_region_hora": (promedio_rh[vacias] - auditoria.loc[vacias, "mwh_real"]).abs(),
})
estrategias.agg(["mean", "median", "max"]).round(2)
```

	interpolar	promedio_region_hora
mean	5.40	27.01
median	2.12	13.56
max	65.97	254.42

Es la respuesta a la pregunta que dejó abierta el ejercicio del bloque 4. Sobre las mismas 1.844 filas, el promedio de la región y la hora se equivoca en 27,01 MWh en promedio y la interpolación en 5,40. El promedio se calcula solo con las filas intactas, para que ninguna estimación nuestra entre en la cuenta.

7.5 Las peores

```
# 7.5
# Las diez filas donde más nos equivocamos.
peores = auditoria.nlargest(10, "error")

# round sobre una tabla con fecha y texto adentro avisa. Se le dice qué
# columnas redondear y las demás quedan como están.
peores[["fecha", "hora", "region", "mwh_real", "mwh_limpia", "error"]].head(5).round(
    {"mwh_real": 1, "mwh_limpia": 1, "error": 1})
```

	fecha	hora	region	mwh_real	mwh_limpia	error
40115	2024-08-26	18	Metropolitana	1228.0	1162.0	66.0
26437	2024-06-06	8	Metropolitana	1131.2	1065.3	65.8
39457	2024-08-22	20	Metropolitana	1242.0	1177.3	64.7
44154	2024-09-19	19	Metropolitana	1196.1	1131.8	64.3
59442	2024-12-19	19	Metropolitana	946.8	888.0	58.8

Las diez filas con más error, todas de la Metropolitana. No es que esa región se haya limpiado peor, es la de mayor magnitud, así que el mismo error relativo se ve más grande en MWh. Por eso conviene mirar también el error relativo.

La solución del ejercicio

El ejercicio pedía el error porcentual con el conteo de cada grupo.

```
print(auditoria.groupby("grupo")["error_pct"].agg(["count", "mean"]).round(2))
```

Con eso queda la frase que se puede poner en un informe. El 3 por ciento de los datos fue estimado por interpolación, menos del uno por mil fue corregido por imposible, y las dos cosas están identificadas fila por fila.

Tres preguntas de repaso

1. Por qué este bloque no se puede hacer en el trabajo real

Porque el dato verdadero no existe, el archivo sucio es el único que hay. Lo que sí queda es la marca de qué filas se tocaron.

2. Qué dice que el error esté concentrado en las filas rellenadas

Que la limpieza no movió lo que no había que mover. Es exactamente lo que uno quiere ver.

3. Por qué las peores filas son todas de la Metropolitana

Porque es la región de mayor magnitud. Una tabla de errores ordenada por valor absoluto siempre está dominada por los grupos grandes, y por eso conviene mirar el error relativo.

Hoja de referencia del Módulo 3

Para imprimir y tener al lado

DIAGNOSTICAR, ANTES DE CALCULAR NADA

<code>df.info()</code>	Tipos, no nulos y memoria
<code>df.shape</code> contra la cuenta teórica	Periodos por unidades, tiene que calzar
<code>df["col"].value_counts()</code>	Variantes de escritura, cola larga sospechosa
<code>df.isna().sum()</code>	Cuántos faltan y dónde

ARREGLAR TIPOS, MEJOR EN LA LECTURA

<code>sep=";", decimal=","</code>	Archivo chileno típico
<code>encoding="utf-8-sig"</code>	Archivo con marca al inicio
<code>pd.to_datetime(col, format="%d/%m/%Y")</code>	Conversión estricta, falla si no calza
<code>col.str.strip().str.lower()</code>	Juntar las variantes de escritura
<code>col.map(CANON)</code>	Dejarlas en la forma oficial

DUPLICADOS

<code>df.duplicated().sum()</code>	Filas repetidas enteras
<code>df.duplicated(subset=CLAVE).sum()</code>	Claves repetidas, se escribe siempre
<code>df.drop_duplicates()</code>	Deja la primera copia

NULOS, LA PARTE QUE ES DECISIÓN

<code>df["fue_nulo"] = df["col"].isna()</code>	Primero la marca, después el relleno
<code>df["fue_corregido"] = sospechosas</code>	Lo mismo con los valores que se cambian
<code>df.sort_values(CLAVE)</code>	Ordenar antes de rellenar por vecino
<code>.groupby(g)["col"].transform(lambda s: s.interpolate())</code>	Siempre dentro del grupo, nunca entre regiones La estrategia que mejor funcionó acá

VALORES IMPOSIBLES Y DESCRIPCIÓN

<code>df[df["col"] < 0]</code>	La regla de negocio pilló lo que ningún método pilló
<code>describe(), median(), skew()</code>	Promedio y mediana siempre juntos
<code>quantile(0.95)</code>	Demanda alta sin quedarse con el máximo

Las cinco cosas que hay que recordar del módulo

- A todo archivo nuevo se le hacen cuatro preguntas antes de calcular.
- Los tipos se arreglan en la lectura, y el formato de la fecha se escribe a mano.
- Los nulos vienen en bloques, y cada estrategia de relleno da un resultado distinto.
- Antes de rellenar se deja la marca de qué filas se tocaron. Es lo único que sobrevive.
- La regla de negocio encuentra lo imposible, que es justo lo que el método sofisticado puede no ver.